

ASSAR Rwanda Insurance Pricing & Information Engine

Design Report: Architecture, Decisions, and Trade-offs

This document describes the ASSAR pricing-and-information system, the engineering decisions taken while building and extending it, and the trade-offs behind each choice. It is written so that a reader who has never seen the codebase can understand both what the system does and why it is shaped the way it is. The source material throughout is the Association of Insurers of Rwanda (ASSAR) Approved General Business Pricing Manual for the Rwandan Insurance Industry, Version 3, effective 25 January 2021, an 81-page document covering minimum premium rates and underwriting guidance for roughly two dozen classes of general (non-life) insurance.

1. Executive Summary

The project turns a long, table-heavy PDF pricing manual into a system that can both compute premiums deterministically and answer natural-language questions about the manual's numbers. It does this with a deliberately hybrid design: exact numeric tables live in SQLite, while the manual's prose (definitions, conditions, warranties, exclusions, underwriting guidance) lives in a vector store for semantic retrieval. A language model never performs the pricing arithmetic itself; it extracts parameters, calls typed Python calculators that read exact rates from the database, and then phrases the result. That separation keeps quotes reproducible and unit-testable while still allowing flexible, conversational questions.

During this engagement the system was extended with a second, purpose-built database designed specifically for a text-to-SQL information engine: one cleanly named SQL table per table in the PDF. This lets an agent map a plain question such as "what is the fire rate for a bank?" directly to a single, well-named table, so an end user can query the manual's contents in natural language instead of reading eighty-one pages. The work was verified against

the source, committed to version control on a feature branch, merged to the main branch, and pushed to the remote repository.

2. Problem and Context

Insurance pricing manuals are reference documents, not databases. The ASSAR manual interleaves dense numeric tables (a 104-row fire-risk grid, commodity transit grids, liability rates, engineering rates, bonds, political-violence rates, large-risk registers) with paragraphs of underwriting prose. Two very different kinds of questions arise from such a document. The first is quantitative and exact: what minimum rate applies to a given risk, what multiplier applies for a chosen indemnity period, what the mandatory excess is. The second is qualitative and semantic: what a clause means, what is excluded, which warranty must be incorporated, how a cover behaves.

A single retrieval-augmented-generation (RAG) pipeline handles the second kind of question well but is a poor fit for the first. Embedding a value such as 0.3144% into a vector and retrieving it by similarity is fragile: numerically adjacent or textually similar cells blur together, and a language model asked to read a rate out of retrieved text can easily transcribe or round it wrong. In insurance, a wrong rate cell is not a cosmetic error; it is a real underwriting and financial error. The core design problem, therefore, is to serve exact numbers exactly while still answering open-ended prose questions fluently.

3. System Architecture

The system is organized around the principle that numbers and prose deserve different storage and different retrieval mechanics.

Numbers are stored in SQLite. SQLite was chosen because it is a single self-contained file with zero server setup, ships inside the Python standard library, supports ordinary SQL, and can be opened by any database browser or by a language model emitting SQL. For a reference dataset of a few hundred rows this is more than sufficient and far simpler than running a database

server.

Prose is stored in a local vector store (ChromaDB) with embeddings computed on-device by a sentence-transformers model. Running embeddings locally avoids sending the manual to a third-party embedding API and removes any per-call cost or network dependency for retrieval. The default embedding model is a small English model chosen for speed on a CPU; a multilingual model is available as a drop-in replacement should Kinyarwanda or French content be added later.

The control flow is a router. A free-text query is inspected; if it is a pricing or number request it is dispatched to typed Python calculators that read exact rates from SQLite and compute the premium deterministically; if it is a concept question it is dispatched to similarity search over the vector store. The language model's role is confined to understanding the request, selecting the right tool, and composing a grounded, cited answer. It does not invent numbers and it does not do arithmetic. A separate quote path bypasses the model entirely and calls the calculators directly, so the pricing functionality works even with no model or API key configured.

The trade-off embodied here is determinism and auditability over end-to-end neural flexibility. A purely generative system would be simpler to wire up but impossible to trust for binding figures. By constraining the model to orchestration and explanation, the system keeps the parts that must be correct under deterministic, testable control, and reserves the model for the parts where fluency and judgement actually help.

4. The Data Layer: Two Databases

A defining decision of this engagement was to maintain two SQLite databases built from the same manual but shaped for two different consumers. They are not redundant; each is optimized for a distinct access pattern, and conflating them would degrade both.

4.1 The Pricing Database

The first database, `assar.db`, is built from a transcription module and uses a

small set of generic, normalized tables. A single rate table holds most per-category rates, namespaced by a scheme column so that fire, public liability, aviation, bonds, and other families coexist in one place. Separate tables hold the two-dimensional transit commodity grids, the ordered discount and multiplier schedules, and per-product constants such as minimum premiums and mandatory excesses. This shape is deliberately tuned for the pricing calculators, which iterate, join, and look up rates programmatically. It is covered by a suite of deterministic tests that pin the arithmetic of the quote functions. Because those calculators and tests depend on this exact schema, the database is treated as stable and is not restructured.

4.2 The Information-Engine Database

The second database, `assar_info.db`, was created during this engagement for a different purpose: a text-to-SQL information engine. Here the design rule is inverted. Instead of a few generic tables namespaced by a column, there is one cleanly named SQL table per table in the PDF: a fire-and-allied-perils table, a special-perils table, a bonds-and-guarantees table, a marine-cargo table, liability tables, engineering tables, the large-risk registers, and so on. Forty-five tables in total, holding several hundred rows.

The reason for this shape is that a language model writing SQL performs far better against descriptive, single-purpose tables than against a generic table keyed by an opaque scheme string. A question such as "what is the rate for a performance bond?" maps naturally to a query against a bonds-and-guarantees table with a bond-type column; the model does not have to know an internal namespacing convention. Well-named tables and columns are, in effect, the schema documentation the model reads to ground its query.

Why Two Databases Rather Than One

The alternative was to force a single schema to serve both the calculators and the query engine. That was rejected. The calculators want compact, normalized tables that are convenient to join and iterate in code; the query engine wants verbose, self-describing tables that read almost like the PDF. Optimizing one schema for both goals would compromise each. Keeping two databases, both

rebuilt from the same verified source data, lets each serve its consumer cleanly. The cost is a second build step and a second file to keep in sync, which is a small and well-contained price. Critically, the new database was added without touching the existing one, so the pricing calculators and their tests were never put at risk.

5. Key Decisions and Trade-offs

This section records the specific judgements made while building and extending the system, and the reasoning behind each.

5.1 Verify Before Rebuilding

When the full PDF first became available, the obvious-looking request was to extract its tables into SQL from scratch. But the project already contained a database built from this very manual. Rebuilding blindly would have duplicated existing, possibly-correct work and risked introducing fresh transcription errors. The chosen course was first to verify the existing data against the newly visible source: spot-checking the cells most prone to error, including the special-perils rates, the fire grid, the per-mille political-violence rates, the bond rates, the aviation rates, and the two-dimensional transit grids. Every checked value matched the source, and the existing test suite passed. Establishing that the underlying numbers were trustworthy meant the later work could reuse them rather than re-transcribe them, which materially reduced risk.

5.2 One Table Per PDF Table

The central decision for the information engine was granularity: how many tables, and how named. The generic four-table schema that serves the calculators is awkward for an agent because it hides each class behind a scheme value. The decision was to emit one cleanly named table per table in the PDF. This maximizes the agent's chance of writing a correct query from a plain question, at the cost of more tables to create and a looser mapping to the calculators. Since the two databases are separate, that looser mapping is acceptable: the information engine does not need to feed the calculators.

5.3 Reuse Verified Values Versus Re-transcribe

Having verified the seed values, the next question was whether to reuse them when building the new tables or to re-transcribe from the PDF. Re-transcription would have reintroduced exactly the transcription risk that verification had just eliminated. The decision was to reuse the verified numeric values programmatically and only transcribe content that was genuinely missing, namely the large-risk registers. This keeps a single source of truth for the numbers and confines new manual transcription to the smallest necessary surface.

5.4 Exact Verbatim Labels

The first build of the information engine normalized category labels into a tidy, title-cased form. On review, this was the wrong call for a query engine: if the stored label differs from the manual's wording, an agent's text matching against the source phrasing can miss. The labels were therefore rebuilt to be exact verbatim strings from the PDF, including punctuation, ampersands, parentheses, and even the typographic ellipsis character. The numeric values remained the verified ones. To guarantee the exact-label lists never drift out of alignment with the verified value lists, a pairing helper asserts equal length at build time and fails loudly otherwise, so a mismatch can never silently shift a label onto the wrong number. The trade-off is that some labels are long and contain unusual characters, which means an agent should use partial matching rather than exact-equality matching; this is a reasonable and well-understood pattern for text-to-SQL.

5.5 Units and a Data Dictionary

A subtle correctness hazard in any rate table is the unit. Most columns already encode their unit in the column name, but a grid of bare numbers can still leave a reader unsure whether a figure is a percentage, a per-mille rate, or a franc amount; and one table of market parameters genuinely mixes francs and percentages in a single value column. Two measures were taken. First, the mixed table gained an explicit per-row unit column. Second, a data-dictionary table was generated that documents the unit of every column across all data tables, so a user or an agent can simply query the units rather than guess. The

political-violence rates are flagged specifically as per-mille, because mistaking a per-mille rate for a percentage is the easiest order-of-magnitude error to make in this manual.

5.6 Transcribing the Large-Risk Registers

The manual's closing pages list named large risks and their insured values across property, engineering, and accident classes. These were not part of the pricing seed, because they are reference data rather than rate inputs, but they are exactly the kind of numeric general information a user might want to query. They were therefore transcribed into their own tables. Insured values were captured as integer franc amounts so they can be sorted and compared numerically, which makes questions such as "what are the largest property risks?" answerable directly.

5.7 A Separate Database File

A final structural choice was where to put the new tables. Adding them to the existing pricing database would have mingled two schemas with two purposes in one file and risked disturbing the calculators. A separate file keeps the concerns cleanly divided, lets each database be rebuilt independently, and follows the repository's existing convention of committing the built database so the project runs out of the box.

5.8 Generating This Report Without Adding Dependencies

This report itself reflects a small but characteristic trade-off. At the moment it was produced, a large dependency installation for the vector store was already running in the background, downloading a substantial machine-learning runtime. Starting a second package installation concurrently risked competing for the same environment and corrupting the in-progress install. Rather than add a third-party PDF library, the report is rendered by a pure standard-library generator that emits the PDF directly. It uses the built-in monospaced core font so that line wrapping is exact without needing font-metric tables, transliterates any unusual characters to a safe set, compresses each page stream, and writes a correct cross-reference table by hand. The trade-off is plainness: the output is clean and readable rather than typographically

elaborate. In exchange it has zero dependencies, cannot conflict with any other installation, and will run on any Python without setup. This mirrors a theme that runs through the whole project: prefer the simplest mechanism that is correct and self-contained over a richer one that adds fragility.

6. Alternatives Considered

Several plausible alternative designs were weighed and set aside, and recording them clarifies why the chosen shape is what it is.

A single pure-RAG system, embedding the entire manual including its tables and answering everything by retrieval and generation, was the most obvious alternative. It was rejected for numbers because embeddings blur exact values and a generative model reading a rate out of retrieved text can round or transcribe it incorrectly. RAG remains the right tool for prose, which is exactly where the design uses it, but it is the wrong tool for binding figures.

A single database serving both the calculators and the query engine was considered and rejected, as discussed above, because the two consumers want opposite schema shapes. A related idea, storing the rates as flat CSV or JSON files rather than in SQLite, was rejected because it would give up SQL querying, sorting, and filtering, which are precisely what a text-to-SQL agent needs, and would gain nothing in simplicity over a single SQLite file.

Fine-tuning a language model on the manual's contents was considered and rejected as both heavier and less trustworthy than retrieval and structured lookup: it would bake the figures into opaque weights, make updates expensive, and still provide no guarantee of numerical exactness. A knowledge-graph representation was considered for the relationships between covers, extensions, and conditions, but it was judged premature; the manual's structure is predominantly tabular, and a graph would add modelling overhead without a clear near-term payoff. These can be revisited if the system grows beyond a single manual.

Finally, having the language model compute premiums directly from retrieved rates was rejected in favour of typed Python calculators. Letting the model do

arithmetic reintroduces exactly the non-determinism the design exists to avoid; confining it to parameter extraction and explanation keeps the financially sensitive computation under tested, reproducible control.

7. The Retrieval Layer

The prose half of the system is fully implemented in code. A corpus builder extracts the manual's narrative text from the PDF into a Markdown corpus, tagging each section with its page so that answers can cite a page. An ingest step chunks that corpus into overlapping passages, embeds them locally, and writes the vectors into a persistent ChromaDB collection. A retriever performs cosine-similarity search to fetch the most relevant passages for a question, which the router then feeds to the language model as grounding.

At the time of writing, the extracted corpus is present but the vector store has not yet been built on this machine, because building it requires installing the embedding and vector-store dependencies and downloading the embedding model on first run. That build is the remaining step to make the retrieval side live; once the dependencies finish installing, ingesting the corpus produces the vector store and the concept-question path becomes operational. The chunking strategy uses a moderate passage size with overlap and prefers to break on sentence or line boundaries, which balances retrieval precision against keeping enough context in each chunk to be self-explanatory.

8. Quality and Verification

Confidence in the system rests on several layers. The pricing calculators are pinned by a deterministic test suite that exercises the quote arithmetic, so a regression in the math is caught immediately. The numeric values were spot-checked against the source PDF across the highest-risk tables before any reuse. The information-engine build includes a structural guard that fails if exact labels and verified values fall out of alignment, which prevents the most likely silent error in that build. After building the new database, sample queries that resemble real agent questions were run and their answers checked

against the PDF: the fire rate for a bank, the rate for a performance bond, the largest property risks by insured value, the marine-cargo rate for pharmaceuticals, and the per-mille political-violence rate for hotels and banks all returned the correct figures. The stored text was also checked at the byte level to confirm that verbatim characters such as the typographic ellipsis were preserved correctly as proper Unicode rather than corrupted, since a console display artifact had initially suggested otherwise.

9. Version Control and Repository Hygiene

The new work was committed deliberately rather than dropped onto the main branch directly. A feature branch was created, the new build module, the built information database, and the updated documentation were staged and committed with a descriptive message, and the branch was then merged into main as a fast-forward and pushed to the remote. The built database was committed intentionally, following the repository's established convention of shipping the database so the project runs without an extra build step; the repository's ignore rules already document this choice.

One hygiene issue was caught and corrected during the work. Internal working notes had initially been written into a folder inside the repository rather than into the assistant's own separate notes location. This would have polluted the project with files that do not belong to it. The notes were moved to their correct location outside the repository and removed from version control before the commit, so the committed history contains only genuine project artifacts.

10. Known Limitations and Risks

The system is a faithful transcription of a single manual, and that framing defines its limits. The rates are only as current as the 2021 manual; if ASSAR revises them, the data must be rebuilt. The values were transcribed by hand at some point in the project's history, and although they have been spot-checked extensively, any binding use should include a deliberate review against the source, because a single wrong cell is a real underwriting error. Two specific

hazards are worth restating: the political-violence rates are expressed in per-mille rather than percent, which is the easiest order-of-magnitude mistake to make; and a small number of band edges in the voluntary-deductible schedule are described with overlapping boundaries in the source, so the engine resolves an exact-boundary value to a defined side and that judgement should be confirmed with ASSAR.

For the information engine specifically, the verbatim labels include long strings and unusual characters, so an agent querying them should rely on partial text matching rather than exact equality, and should consult the data-dictionary table for units rather than assuming a default. The retrieval layer is not yet live until its vector store is built, so concept questions are unavailable until that step completes.

11. Recommended Next Steps

The most immediate step is to finish building the vector store so the prose side of the system becomes operational, completing the hybrid design. After that, the natural next piece is the agent layer that ties everything together: a router that classifies an incoming question, sends number questions to the information database as SQL and concept questions to the retriever, and composes a single grounded answer with citations. The data-dictionary table should be supplied to that agent as part of its schema context so it always knows the unit of every figure it returns.

Beyond that, a light validation harness for the information database, analogous to the pricing tests, would guard the new tables against future edits, and a periodic check against any revised ASSAR manual would keep the data current. Together these steps would turn a static reference PDF into a trustworthy, queryable, conversational pricing and information service: deterministic where the numbers must be exact, fluent where the questions are open-ended, and auditable throughout.

Appendix A: Information-Engine Table Catalog

The information database contains forty-five tables. They are grouped here by theme to convey the breadth of coverage; column units are documented in full by the data-dictionary table that accompanies them.

Property and fire cover is served by a fire-and-allied-perils table holding the full commercial and administrative risk grid with its standard-fire and all-perils columns, a fire-private-dwellings table, a special-perils table giving the commercial-industrial and residential rate for each peril, and a plate-glass table. Business-interruption cover is served by a consequential-loss basis table, a consequential-loss indemnity-period multiplier table, and a shared time-excess discount table.

Theft and money cover is served by a burglary full-value table, a first-loss multiplier table, a money-insurance table covering transit and safe, and a money annual-carryings band table. The financial-institution covers are served by a bankers-blanket-bond table and a directors-and-officers-liability table.

Transit cover is served by three structurally similar grids: goods-in-transit, transporters-liability, and marine-cargo, each keyed by the manual's commodity classification codes and carrying the containerized and non-containerized rate columns and the applicable excess. Marine and aviation cover add a marine-hull table, a marine-hull per-occupant premium table, and an aviation table.

The liability suite comprises public-liability, employers-liability, product-liability, professional-indemnity, and a combined personal-accident and group-personal-accident table, with school-liability captured separately including its benefit limits. Engineering cover is served by erection-all-risks, contractors-all-risks, machinery-breakdown, a contractors-plant-and-machinery rate table keyed by hazard class and plant group, a boilers-and-pressure-vessels table, and a computer-and-electronic-equipment all-risks table.

Guarantees and special risks are served by a bonds-and-guarantees table, a fidelity-guarantee table, and a political-violence-and-terrorism table whose rates are expressed in per-mille. Several schedule tables capture the voluntary-deductible discounts, the short-period scales for the standard case and for schools, personal accident, and contractors plant, and the indemnity-period multipliers. Reference tables capture the three large-risk

registers for property, engineering, and accident classes; a market-parameters table holding policy fee, commission, and discount caps with explicit per-row units; a minimum-premiums table; and the data dictionary that documents the unit of every column in the database.

This catalog is intended to be read by a human, but its real audience is the agent: supplied as schema context, these descriptive table and column names are what let a plain-language question be turned into a correct query.

End of report.